PreNS: A Hybrid Predictive and Real-Time Resource Allocation Framework for 5G and Beyond RAN Network Slicing

Binghui Wu, Palleri Chettuparambil Amogh, Nalam Venkata Abhishek, *Member, IEEE*, and Mohan Gurusamy, *Senior Member, IEEE*

***Abstract*—The rapid evolution of 5G and beyond-5G (B5G) networks, coupled with the emergence of network slicing, has introduced unprecedented challenges in resource allocation for Radio Access Networks (RAN). Existing methods primarily fall into two categories: 1) reactive approaches, which fail to anticipate sudden traffic changes; 2) predictive methods, which struggle to adapt to real-time conditions. These limitations result in inefficient resource utilization, degraded Quality of Service (QoS), and increased latency. To address these challenges, we propose PreNS, a hybrid framework that integrates predictive modeling with real-time decision-making to optimize resource allocation. PreNS combines an attention-based Bi-LSTM model for accurate traffic demand forecasting with a Double Deep Q-Network (DDQN) for real-time adjustments. Its key innovation lies in the aggregated state representation, which fuses predicted demands and real-time data, enabling adaptive and efficient resource management. By balancing proactive foresight and reactive adaptability, PreNS addresses fluctuating network conditions and the rigidity of static optimization. Experimental evaluations on a realistic 5G testbed demonstrate that PreNS improves resource utilization by approximately 14%, reduces latency by 20%, and enhances QoS satisfaction by 5%, compared to state-of-the-art methods—demonstrating its potential for efficient, scalable resource allocation in future wireless networks.**

***Index Terms*—5G and beyond, radio access networks, network slicing, proactive resource allocation, deep learning.**

I. Introduction

ADVANCEMENTS in communication technologies have catalyzed the growth of diverse vertical services and applications, each with stringent Quality of Service (QoS) requirements [1]. To meet these demands, fifth-generation (5G) networks introduced *network slicing*, which partitions a single physical network into multiple virtual slices, each tailored to specific service needs [2], [3]. As we move toward Beyond 5G (B5G) and sixth-generation (6G) networks, resource allocation becomes increasingly complex due to the proliferation of smart city applications and Internet of Things (IoT) devices [4]. Many existing 5G slicing frameworks fall short in meeting key performance indicators (KPIs), such as ultra-low latency, massive connectivity, and reliable resource allocation. This gap becomes especially pronounced in highly dynamic, unpredictable environments [5]. Traditional resource allocation methods struggle to adapt to the fast-changing and diverse traffic patterns in B5G networks, resulting in inefficient resource utilization, degraded QoS, and increased latency [6], [7]. In response, machine learning (ML) and deep learning (DL) approaches, especially deep reinforcement learning (DRL), have shown promise in this domain [8], [9]. They are primarily reactive, relying solely on current network data and failing to anticipate sudden traffic surges. On the other hand, predictive models using techniques like ARIMA [10], Holt-Winters [11], CNNs [12], [13], and RNNs [14], [15], [16], [17] aim to forecast traffic trends but often ignore real-time deviations, limiting their effectiveness in volatile conditions.

Most existing resource allocation frameworks for 5G and beyond networks fall into one of two extremes: i) purely reactive, relying entirely on real-time network observations; ii) purely predictive, depending entirely on traffic forecasts. However, reactive approaches respond effectively to current conditions but struggle to anticipate sudden demand spikes. Predictive approaches, on the other hand, can plan ahead but often fail to adapt when real-time conditions deviate from the forecast, a common occurrence in dynamic environments like ultra-reliable low-latency communications (uRLLC) or emergency services.

Critical gap: Existing methods lack mechanisms to dynamically evaluate and adapt the trustworthiness of predictions. In practice, forecast accuracy degrades under sudden traffic shifts or mobility changes. However, current systems either fully trust predictions, ignoring real-time deviations, or discard predictions altogether. A more robust strategy must balance predictive foresight and real-time observations based on their current reliability. This requires a *confidence-aware aggregation* of forecasts and live network metrics, enabling proactive resource allocation when predictions are accurate and

Received 26 December 2024; revised 29 April 2025, 9 September 2025, 6 November 2025, and 20 January 2026; accepted 6 April 2026. Date of publication 13 April 2026; date of current version 23 April 2026. This research is supported by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Communications and Connectivity Bridging Funding Initiative. The associate editor coordinating the review of this article and approving it for publication was Y. Liu. *(Corresponding author: Binghui Wu.)*

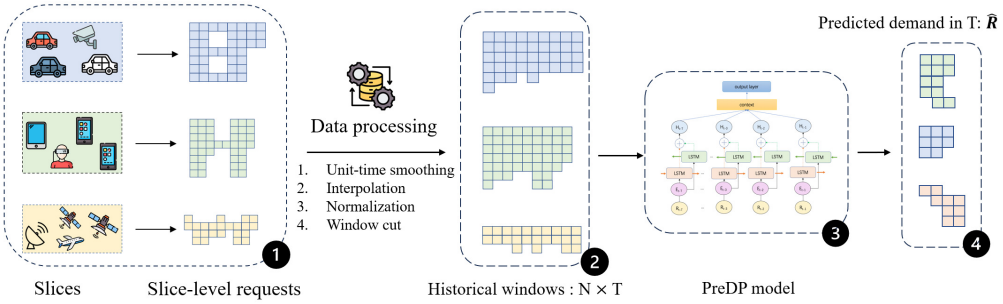
Binghui Wu and Mohan Gurusamy are with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117583 (e-mail: binghuiwu@u.nus.edu; gmohan@nus.edu.sg).

Palleri Chettuparambil Amogh and Nalam Venkata Abhishek are with the Infocomm Technology Cluster, Singapore Institute of Technology, Singapore 828608 (e-mail: amogh.pc@singaporetech.edu.sg; nvenkataabhishek@gmail.com).

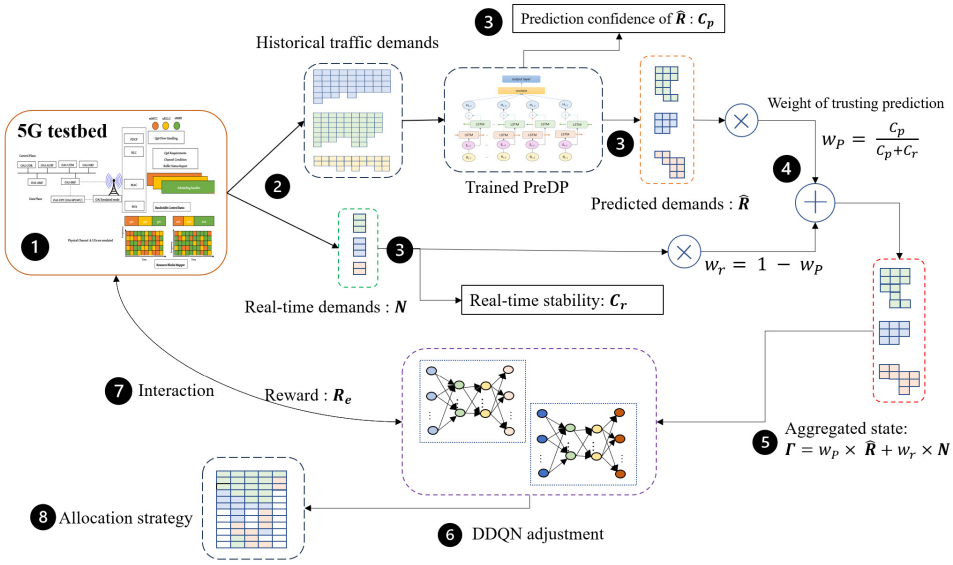
Digital Object Identifier 10.1109/TCCN.2026.3683197

2332-7731 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.



(a) PreDP prediction of future demands



(b) DDQN dynamic allocation

Fig. 1. PreNS architecture illustration.

flexible fallback to real-time adaptation when deviations occur. Despite its importance, such trust-adjusted fusion strategies have received little attention in prior work.

Recognizing these limitations, we propose **PreNS**, a hybrid architecture for adaptive resource allocation in 5G-and-beyond RAN environments. As illustrated in Figure 1, which depicts the data flow and module interactions in PreNS, the framework leverages a *confidence-aware aggregated state* that combines predictive insights with real-time network observations to guide intelligent allocation decisions. PreNS comprises two key components: i) the PreDP prediction module and ii) the DDQN-based decision module. The PreDP module employs a Bi-directional Long Short-Term Memory (Bi-LSTM) network with an attention mechanism to forecast short-term traffic demands. The Bi-LSTM captures temporal dependencies more effectively than unidirectional models [18], while attention improves forecasting accuracy by highlighting relevant temporal features [19]. Forecasts are fused with current traffic levels, resource availability, and QoS metrics to form a confidence-weighted aggregated state, which is dynamically updated based on the reliability of past predictions and real-time stability. With this aggregated information in hand, the DDQN module adaptively allocates resources across network slices, enabling proactive adjustments when predictions are reliable and reactive responses under uncertain conditions.

By jointly considering historical and real-time context, PreNS enables fine-grained, low-latency resource management that is both predictive and responsive—key to meeting the heterogeneous QoS requirements of B5G networks. The main contributions of this work are summarized as follows:

1) **Confidence-aware state aggregation:** We propose a mechanism to dynamically integrate predicted and real-time network features, weighted by confidence scores (Section IV-B). This enables context-aware and trust-adjusted decision-making in volatile environments.
2) **Hybrid framework for predictive and real-time resource management:** PreNS integrates a Bi-LSTM-based predictor and a DDQN-based controller to support both foresight and responsiveness (Section IV-C). This overcomes the limitations of prior methods that rely exclusively on either prediction or real-time metrics.
3) **Enhanced QoS and resource efficiency:** By leveraging the aggregated state, PreNS improves resource utilization and maintains consistent QoS guarantees across heterogeneous slices, including latency-sensitive and bursty applications (Section VII).

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

4) **Realistic testbed implementation and evaluation:** We implement a 5G testbed using open-source tools to emulate diverse traffic scenarios and validate the robustness of PreNS under practical conditions (Section V). The code will be publicly released upon acceptance.

The remainder of the paper is structured as follows: Section II reviews related work on network slicing and resource allocation. Section III presents the system model and problem formulation. Section IV details the PreNS architecture. Section V describes the experimental testbed. Section VII provides evaluation results. Finally, Section VIII concludes the paper and outlines future research directions.

II. Related Work

A. Demand Forecasting

Predictive resource allocation has gained significant attention in 5G and B5G networks as a way to address the limitations of purely reactive resource management. By forecasting future traffic demands, predictive models enable proactive resource provisioning, allowing networks to anticipate demand surges and allocate resources more efficiently [20]. Traditionally, non-ML solutions such as ARIMA [10] and Holt-Winters [11] have been widely used for temporal forecasting. However, these methods fall short in capturing the complex patterns of mobile traffic, particularly in scenarios involving user mobility.

Consequently, various ML/DL-based forecasting methods have been explored as more effective alternatives. DeepCog [13], for example, employs a 3D-CNN model to forecast network demand from historical antenna-level traffic, with a cost-aware loss function to reduce over-provisioning and SLA violations. However, CNNs, originally designed for vision tasks, struggle to capture temporal dependencies [21]. Since network demand is inherently sequential, Recurrent Neural Networks (RNNs) provide an advantage. LSTM models, in particular, are widely adopted for traffic prediction due to their ability to learn long-term dependencies. Greff et al. [22] demonstrate that LSTM can accurately forecast mobile traffic patterns, improving resource allocation. Similarly, Azimi et al. [8] combine LSTM-based prediction with RL for allocation decisions, though this hybrid design increases system complexity and fails to meet stringent low-latency requirements in B5G. Extensions such as Bi-LSTM with attention [23] improve accuracy but remain less responsive to abrupt traffic shifts. GRU-based models [15] offer a lighter alternative, delivering accuracy comparable to LSTM while reducing computational overhead, thus underscoring the trade-off between efficiency and accuracy.

B. Resource Allocation in Network Slicing

Resource allocation can be either coarse-grained or fine-grained [24]. In a coarse-grained approach, resources are allocated to network slices based on average slice-level QoS requirements, such as throughput and latency, without considering individual end-user needs [25]. In contrast, a fine-grained approach allocates resources like RBs, CPU, and transmit power to each slice's users (UEs), ensuring their QoS satisfaction. Recently, many studies have employed machine learning techniques to address the complexities of resource allocation in network slicing. Deep Reinforcement Learning (DRL) has emerged as a powerful tool for resource allocation in 5G networks due to its ability to learn optimal policies in complex and dynamic environments [26], [27], [28], [29], [30]. Recent work suggests that combining coarse- and fine-grained resource allocation yields improved utility [30], [31], [32], [33]. One of the earliest DRL applications for resource allocation was introduced in [34], where a DQN-based method was proposed to optimize RB allocation in 5G RAN, maximizing access rates while minimizing costs. A major limitation of DQN, however, is overestimation bias, which can lead to suboptimal decisions in highly stochastic environments such as network slicing. To address this, [33] employed DDPG for two-level allocation, showing higher slice utility than DDQN and PG. Other works explore multi-timescale allocation, combining coarse-grained and fine-grained resource management. For example, [32], [35] propose a collaborative framework using LSTM-based traffic forecasting for large-timescale allocation and RL (A3C or DDPG) for real-time RB allocation, outperforming baseline RL methods without forecasting in terms of RB utilization and cumulative reward. In [36], a double-layer framework divides allocation into base station–to–slice and slice–to–user processes, using a DRL algorithm with parameter tuning to improve responsiveness and user satisfaction. Similarly, [37] apply recurrent deterministic policy gradient (RDPG), leveraging historical context for memory-based resource allocation under uncertainty in network slicing.

In summary, while prior work has explored either predictive or DRL-based approaches for resource allocation, few have effectively combined the two in a confidence-aware manner. Existing methods either fully trust predictions or ignore them, lacking a mechanism to adaptively balance foresight and real-time feedback. Besides, our work targets a single-base-station scenario with a discrete action space and strict latency requirements. In this setting, the DDQN framework offers more stable convergence and lower training variance compared to asynchronous policy-gradient methods. In contrast, algorithms such as A3C [32] and PPO [35], [37] are better suited for multi-agent or federated RAN slicing scenarios, where asynchronous updates and continuous control can improve scalability across distributed edge nodes. Therefore, our design focuses on enhancing state representation and adaptive fusion within the DDQN framework, which can later be extended to multi-agent settings.

III. System Model and Problem Formulation

This section presents the system model for dynamic resource allocation, focusing on managing resources for network slices $\mathcal{O} = \{1, 2, \ldots, O\}$. The network comprises resource blocks $\mathcal{R}$, shared among these slices. Each slice $o \in \mathcal{O}$ supports a set of mobile users, $\mathcal{U}_o = \{u_o^1, u_o^2, \ldots, u_o^{|\mathcal{U}_o|}\}$, and requires resources such as bandwidth, computing power, and server capacity to meet its users' QoS needs. The number of users in slice o is $|\mathcal{U}_o|$, with u_o^m denoting the mth user. Because

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

resource demands vary with slice type and user behavior, our goal is to develop an efficient, predictive mechanism that dynamically allocates bandwidth and other essential resources (e.g., computing power). Consequently, allocation occurs at the base station $\mathcal{B}$, which manages resource blocks (RBs) based on user demand and real-time network conditions. In this context, an RB, as defined by the 5G NR standard [38], is the smallest radio resource unit, comprising 12 adjacent Orthogonal Frequency Division Multiplexing (OFDM) subcarriers with a 15 kHz spacing and a 1 ms slot duration. RBs are allocated dynamically, allowing each slice to receive a share of resources based on its priority, traffic requirements, and service-level agreement (SLA).

Each network slice $o \in \mathcal{O}$ serves a specific group of users and supports distinct services, such as eMBB, URLLC, and mMTC. These slices often have different QoS requirements, particularly in terms of: i) *Throughput Requirement* (τ_o), the minimum aggregate data rate required by slice o to meet user demands; and ii) *Latency Requirement* (ξ_o), the maximum allowable end-to-end delay for services in slice o. The achieved throughput and latency for each user u_o^m must satisfy the following constraints:

$$T_o^m \geq \tau_o^m, \quad L_o^m \leq \xi_o, \tag{1}$$

where T_o^m and L_o^m denote the throughput and latency experienced by user u_o^m, and τ_o^m and ξ_o represent the required minimum throughput and maximum acceptable latency, respectively. To manage slices under fluctuating traffic demands, the base station dynamically allocates RBs at two hierarchical levels: slice-level and user-level. This two-tier approach ensures that each slice receives a suitable proportion of RBs, and that users within a slice are served according to their QoS requirements:

1) **Slice-Level Allocation:** The base station allocates RBs to slices based on traffic demand and QoS requirements. This allocation is updated periodically, with intervals ranging from seconds to minutes depending on the service type. Highly dynamic services (e.g., AR/VR, gaming) require more frequent updates, while delay-tolerant services (e.g., mMTC) may operate on longer intervals. The allocation must satisfy:

$$\sum_{o=1}^{O} R_o \leq \mathcal{R}, \quad w_o = \frac{R_o}{\mathcal{R}} \tag{2}$$

where R_o is the number of RBs allocated to slice o, and w_o denotes the proportion of RBs assigned to slice o.

2) **User-Level Allocation:** Within each slice, the allocated RBs are further distributed among the users. This allocation is adjusted in real time to reflect changing user demands, channel quality, and mobility. Let R_o^m represent the number of RBs allocated to user u_o^m in slice o, and $|\mathcal{U}_o|$ denote the number of users in slice o. The following constraints must be satisfied:

$$\sum_{m=1}^{|\mathcal{U}_o|} R_o^m \leq R_o, \tag{3}$$

$$w_o^m = \frac{R_o^m}{R_o}, \tag{4}$$

where w_o^m is the ratio of RBs assigned to user u_o^m relative to the total RBs allocated to slice o.

This work focuses primarily on slice-level resource allocation, which represents the key challenge in dynamic network slicing. User-level scheduling within each slice is handled by the default configuration provided by the testbed (Section V), which we adopt based on its optimized design.

A. Hybrid Allocation Approach

PreNS adopts a *hybrid resource allocation strategy* that combines scheduled updates with event-driven triggers to ensure both efficiency and responsiveness:

1) **Periodic Updates:** The base station regularly adjusts slice-level resource allocations based on predicted traffic demand. These updates occur every *few seconds* for highly dynamic slices and every *few minutes* for more stable ones. Forecasts are generated using a Bi-LSTM-based predictor, enabling proactive adaptation to expected variations in user demand.
2) **Event-Driven Triggers:** Whenever real-time conditions deviate significantly from the predicted patterns—such as sudden traffic spikes—an event flag is raised. This flag prompts the *next* allocation round to occur sooner or with higher priority than usual, allowing critical slices (e.g., uRLLC) to receive additional resources without waiting for the standard scheduling interval. Although this does not reallocate resources instantaneously, it enables an expedited response, ensuring urgent QoS requirements are met under volatile traffic conditions.

B. Utility Functions

Each user u_o^m in the network is associated with a utility function $\Gamma(u_o^m)$, which quantifies the benefit derived from the allocated resources. The function is designed to reward higher throughput while ensuring compliance with slice-level QoS constraints:

$$\Gamma(u_o^m) = \phi_o^m \cdot T_o^m \tag{5}$$

$$\sum_{m=1}^{|\mathcal{U}_o|} \phi_o^m = 1, \quad \phi_o^m \geq 0 \tag{6}$$

Here, T_o^m is the achieved throughput of user u_o^m, determined by the allocated resource blocks R_o^m and channel conditions. The priority weight ϕ_o^m reflects factors such as SLA criticality or latency sensitivity. Users in latency-critical slices (e.g., uRLLC) receive higher weights to ensure timely allocation, while less time-sensitive slices (e.g., eMBB) tolerate modest delays under congestion. The utility of a slice is then defined as the aggregate utility of its users:

$$\Phi_o = \sum_{m=1}^{|\mathcal{U}_o|} \Gamma(u_o^m) \tag{7}$$

and the total network utility is the sum of utilities across all slices:

$$\Phi = \sum_{o=1}^{O} \Phi_o = \sum_{o=1}^{O} \sum_{m=1}^{|\mathcal{U}_o|} \Gamma(u_o^m). \tag{8}$$

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

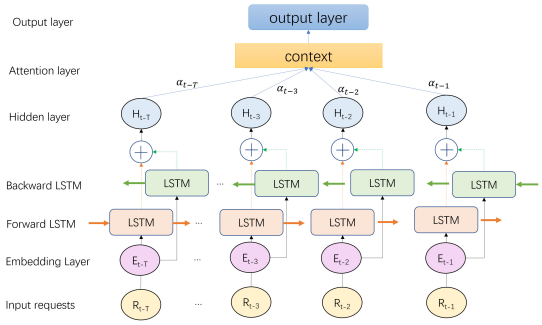


Fig. 2. The architecture of PreDP.

C. Problem Formulation

The objective of the resource allocation problem is to maximize the total network utility Φ, subject to the QoS requirements of each slice. This is formalized as:

$$\max_{\{R_o, R_o^m\}} \quad \Phi = \sum_{o=1}^{O} \sum_{m=1}^{|\mathcal{U}_o|} \phi_o^m \cdot T_o^m \tag{9}$$

$$\text{s.t. constraints} \quad (1) - (6) \tag{10}$$

Solving this optimization problem in real time is computationally challenging due to several factors: the dynamic nature of user demands, the nonlinear structure of utility functions, and the combinatorial explosion of decision variables as the number of slices and users increases. These factors make conventional optimization techniques unsuitable for latency-sensitive 5G network environments.

IV. PreNS Architecture

This section outlines the overall architecture of PreNS, a hybrid learning-based framework that combines predictive modeling with real-time adjustments for adaptive resource allocation in 5G RAN. After describing how PreDP forecasts future demands, we show how confidence-aware aggregation merges prediction with real-time data, and finally demonstrate how the DDQN agent uses this aggregated state for hybrid resource allocation.

A. Demand Prediction: PreDP

This section describes the **demand forecasting module** of PreNS, referred to as PreDP. This module enables the system to anticipate future slice-level traffic demands and allocate resources proactively. By forecasting demand trends over a short time horizon, PreDP allows PreNS to make informed decisions that account for both upcoming surges and low-activity periods, improving overall resource efficiency.

1) Model Overview and Forecast Objective: PreDP is implemented as an attention-based Bi-LSTM model. For each network slice, the model predicts future resource demand $\hat{R}_i(t+k)$ over a fixed prediction window $k = 1, 2, \ldots, T$, using historical traffic observations. The high-level architecture of PreDP is shown in Figure 2.

2) Input Representation: Each slice is modeled independently using its own historical traffic. At each time step t, the input to PreDP is a sequence of the last $N = 5$ normalized traffic demand values, sampled at 100 ms intervals (i.e., over a 500 ms window):

$$X_i(t) = [N_i(t{-}500\text{ms}), N_i(t{-}400\text{ms}), \ldots, N_i(t{-}100\text{ms})]$$

No exogenous variables (e.g., number of users or latency) are included. All inputs are normalized and smoothed as described in Section V.

3) Bi-LSTM Model: The Bi-LSTM model processes the input sequence in both temporal directions:

- *Forward LSTM:* Processes the input from oldest to most recent time step (i.e., from $t{-}500$ms to $t{-}100$ms), generating forward hidden states $\overrightarrow{h}_i$.
- *Backward LSTM:* Processes the input in reverse (from $t{-}100$ms to $t{-}500$ms), generating backward hidden states $\overleftarrow{h}_i$.

At each time step i, the two directional outputs are concatenated to form the combined hidden state:

$$h_i = [\overrightarrow{h}_i; \overleftarrow{h}_i]$$

This results in a sequence of N context-rich hidden states: $H = [h_1, h_2, \ldots, h_N]$

4) Attention Mechanism and Output Prediction: To identify which time steps are most relevant for forecasting future demand, we apply an attention mechanism over the hidden state sequence H. Specifically, the attention mechanism computes a relevance score e_i for each hidden state h_i using a trainable feed-forward layer. These scores are then normalized via a softmax function:

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{N} \exp(e_j)}, \quad e_i = \text{score}(h_i)$$

The resulting attention weights α_i determine the relative importance of each time step. A context vector $z(t)$ is computed as a weighted sum of the hidden states:

$$z(t) = \sum_{i=1}^{N} \alpha_i \cdot h_i$$

This context vector $z(t)$ summarizes the most informative temporal features and is passed through a fully connected layer to generate the predicted resource demand:

$$\hat{R}_i(t) = f(z(t))$$

The attention-enhanced output enables PreDP to focus on informative patterns (e.g., bursty behaviors) while ignoring less relevant noise, thereby improving both forecast accuracy and model interpretability.

5) Robustness Under Bursty Traffic: To evaluate PreDP under highly dynamic conditions, we model a uRLLC slice that issues resource requests every 5 ms, mimicking event-driven behavior (e.g., emergency response). As shown in Section VII, PreDP maintains low prediction error across time, demonstrating resilience to abrupt traffic changes. In the next subsection, we explain how PreNS dynamically balances these forecasts with live network observations to mitigate errors in highly volatile scenarios.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

B. Confidence-Aware State Aggregation

A core challenge in adaptive resource allocation is determining how much to rely on predicted demand versus real-time network observations, especially in highly dynamic conditions. PreNS addresses this by introducing a *confidence-aware state aggregation* mechanism that adaptively balances these two sources based on their current reliability. Rather than treating predicted and real-time values equally, PreNS computes dynamic fusion weights that reflect how much trust to place in each at a given moment. This enables robust, slice-level decision-making under uncertainty.

1) Confidence Metrics Computation: At each decision step t, PreNS computes confidence scores to quantify the reliability of two data sources, predictions and real-time metrics. This mechanism ensures that when forecast accuracy degrades or traffic becomes highly volatile, PreNS naturally reduces its reliance on predicted values.

Prediction Confidence $C_p(t)$ quantifies recent forecasting accuracy. It is defined as the inverse of the average prediction error over a window of size T:

$$C_p(t) = \frac{1}{\epsilon + \frac{1}{T}\sum_{j=t-T}^{t-1} \|\hat{R}(j) - R(j)\|}$$

where $\hat{R}(j)$ and $R(j)$ are the predicted and actual resource demands at time j, respectively, $\|\cdot\|$ denotes an element-wise error norm (e.g., L_1), and ϵ prevents division by zero. A higher $C_p(t)$ indicates more reliable predictions.

Real-Time Stability $C_r(t)$ measures the volatility of recent real-time observations. It is computed as the average variance of network metrics across the same window:

$$C_r(t) = \frac{1}{T}\sum_{j=t-T}^{t-1} \text{Var}(N(j))$$

where $N(j)$ is the real-time observation vector (e.g., resource usage, user count, QoS metrics). Lower $C_r(t)$ implies greater stability and hence higher trust in real-time data.

2) Dynamic Weight Adjustment: The confidence scores are then used to derive normalized fusion weights for each slice:

$$w_p(t) = \frac{C_p(t)}{C_p(t) + C_r(t)}, \quad w_r(t) = 1 - w_p(t)$$

These weights are computed element-wise for each slice, allowing PreNS to emphasize predictions when forecasts are accurate and the network is stable, or lean on real-time data when volatility or prediction error increases. This dynamic weighting mechanism is especially effective for bursty slices (e.g., uRLLC), which may suddenly deviate from forecasted patterns. By adjusting trust on the fly, the system avoids overcommitting to either source.

3) Aggregated State Representation: Using the computed weights, PreNS constructs an aggregated state vector $\Gamma(t)$ that combines predicted demand $\hat{R}(t)$ and real-time data $N(t)$:

$$\Gamma(t) = w_p(t) \odot \hat{R}(t) + w_r(t) \odot N(t)$$

Here, $\odot$ denotes element-wise multiplication. All vectors are of length n, representing the n active network slices at time t. Each element $\Gamma_k(t)$ thus reflects a weighted blend of foresight and current observation for slice k, allowing the system to adapt resource decisions to evolving traffic conditions. The aggregated state $\Gamma(t)$, together with confidence scores $C_p(t)$ and $C_r(t)$, forms the input to the DDQN agent.

C. Hybrid Dynamic Resource Allocation

To implement the hybrid resource allocation approach, PreNS employs a DDQN agent that periodically updates allocations while also responding to event-driven triggers (Section III-A). The agent learns to map network states to optimal resource allocation actions by interacting with a simulated environment that reflects real-time and predicted demands. The decision process relies on three components: the *state* representation, the *action space*, and a *reward function* designed to promote both performance and fairness across heterogeneous slices.

1) State Representation: At each decision time t, the DDQN agent receives a state vector $\mathbf{S}(t)$ that captures both current network conditions and anticipated future demands. This state is composed of:

- $\Gamma(t)$: the confidence-weighted aggregation of predicted demands $\hat{R}(t)$ and real-time observations $N(t)$, computed as:
$$\Gamma(t) = w_p(t) \odot \hat{R}(t) + w_r(t) \odot N(t)$$
- $C_p(t)$: the vector of prediction confidence scores, reflecting recent forecasting accuracy.
- $C_r(t)$: the vector of real-time stability scores, capturing volatility in recent observations.

Thus, the full state input is:

$$\mathbf{S}(t) = [\Gamma(t), C_p(t), C_r(t)]$$

This formulation enables the agent to reason over both the magnitude and reliability of upcoming demands, making it robust to fluctuations or sudden traffic bursts.

2) Action Space: The action $A(t)$ denotes the slice-level allocation decision at time t, i.e., how available resources should be distributed across n slices:

$$A(t) = [a_1(t), a_2(t), \ldots, a_n(t)], \quad \sum_{i=1}^{n} a_i(t) \leq R_{\text{total}}$$

Each $a_i(t)$ lies within predefined bounds $[a_i^{\min}, a_i^{\max}]$, which are derived from SLA constraints or system limitations. The action space is discretized to ensure tractability.

3) Reward Function Design: To guide the learning process, we define a reward function that balances utility and QoS adherence:

$$\text{Re}(t) = \sum_{i=1}^{n} \left[\alpha \cdot U_i(t) - \beta \cdot P_i(t)\right],$$

where $U_i(t)$ reflects the utility gained by slice i, based on QoS compliance, throughput, and user satisfaction; $P_i(t)$ captures penalties from SLA violations, such as latency breaches or under-provisioning; and α, β are tuning parameters controlling the trade-off between performance and constraint adherence.

The DQN comprises two fully connected layers with 128 and 64 neurons, respectively, using ReLU activations, followed

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

by a linear output layer. The agent is trained once per allocation cycle (200 ms), with the target network updated every 200 steps. In addition to periodic updates, the system performs event-based reallocation when real-time demand significantly deviates from predicted values. Specifically, an update is triggered when the deviation between observed and predicted demand exceeds 20% for at least two consecutive allocation cycles. This threshold is chosen to balance responsiveness and stability. An ϵ-greedy exploration strategy is employed with ϵ decayed from 1.0 to 0.05 over 10,000 steps. Rewards are normalized to the range $[-1, 1]$ to stabilize training. The final weights are set to $\alpha = 0.6$ (utility term) and $\beta = 0.4$ (penalty term), selected after grid search (see Table II).

Additionally, each slice is guaranteed a minimum resource share at runtime to avoid starvation. To prevent starvation of low-priority slices, we incorporate a fairness-aware penalty into the reward function. This mechanism enforces a minimum guaranteed share of resources for each slice. When the allocation for slice i falls below its guaranteed floor $\mu_{\min}$, a penalty is applied:

$$P_i(t) = \gamma \cdot \max\left(0, \frac{\mu_{\min} - \mu_i(t)}{\mu_{\min}}\right),$$

where $\mu_i(t)$ is the allocated resource share of slice i, and γ controls the severity of the penalty. The weight γ was tuned empirically to balance fairness and overall performance.

It is important to note that the penalty itself does not encode slice priorities. Priorities are instead reflected in the utility term $U_i(t)$, where stricter QoS requirements (e.g., URLLC latency constraints) yield higher rewards. The fairness penalty guarantees that all slices receive at least a baseline allocation, thereby preventing starvation, while the utility-driven rewards naturally allow higher-priority slices to be favored when additional capacity is available.

4) State Transition Dynamics: The environment models the transition from state $\mathbf{S}(t)$ to $\mathbf{S}(t{+}1)$ as a consequence of the agent's action $A(t)$. Executing $A(t)$ updates the resource allocation, which in turn modifies real-time metrics $N(t{+}1)$ such as utilization and QoS performance. The prediction module (PreDP) then generates updated forecasts $\hat{R}(t{+}1)$ based on recent observations, while the confidence scores $C_p(t{+}1)$ and $C_r(t{+}1)$ are recalculated to reflect prediction accuracy and network stability. This feedback loop enables the agent to continuously adapt its policy in response to both forecasted and observed changes in network dynamics.

D. Training Procedure

The agent employs standard reinforcement learning mechanisms—experience replay, a target network, and an ϵ-greedy exploration strategy—to ensure stable and effective policy learning. Past transitions $(\mathbf{S}(t), A(t), R(t), \mathbf{S}(t{+}1))$ are stored in a FIFO buffer of size 20,000 and uniformly sampled to improve generalization. The target network, updated with a delay relative to the main Q-network, mitigates value overestimation and stabilizes training. Exploration is guided by an ϵ-greedy strategy, which encourages exploration in early stages and gradually shifts toward exploitation as training progresses.

Mini-batches sampled from the replay buffer are used to update the Q-network via gradient descent, with the loss defined as the temporal-difference (TD) error between predicted and target Q-values. The target network is periodically synchronized with the main Q-network to prevent instability. Training is performed on diverse, randomized traffic patterns with balanced transitions across slice types. After convergence, the learned policy is deployed for online slice-level resource allocation in the 5G RAN environment.

V. Testbed Simulation

In this section, we present the testbed simulation and experimental settings of our proposed PreNS framework. The required data is obtained through an in-house emulator. The Network Slicing Framework is implemented using OpenAir-Interface (OAI-5G), a reliable open-source platform compliant with 3GPP requirements for a real-time 5G testbed [39]. The 5G core network control plane components include the Unified Data Repository (OAI-UDR), Authentication Server Function (OAI-AUSF), Unified Data Management (OAI-UDM), Network Function Repository Function (OAI-NRF), Access and Mobility Management Function (OAI-AMF), Session Management Function (OAI-SMF), and User Plane Function (OAI-UPF). These components manage various essential control and data plane functions of the 5G core. User Equipment (UE) and the gNodeB (gNB) component of the Radio Access Network (RAN) are both in simulation mode. To simulate 5G core and gNB, we use OAI-5G Docker containers. Besides, we employ ETSI-aligned cloud-native frameworks known as Kubernetes (K8s) [40] and Prometheus [41] for NFV management and orchestration (NFV MANO). Such frameworks are beneficial for automating deployment, management of containerized dockers, and data collection.

The physical infrastructure of Kubernetes is built on top of commodity hardware servers. It runs Ubuntu 18.04 and has an Intel(R) Xeon(R) CPU E5-2630, 32 cores, and 64GB of RAM. We implement mMTC, uRLLC, and eMBB slices. The Packet Data Convergence Protocol (PDCP), Radio Link Control (RLC), Physical layer (PHY), and Medium Access Control (MAC) are shared by each slice, allowing for a dynamic shared protocol stack in the RAN and less resource waste. The Core Slice Orchestrator (CSO) within the NFV MANO Kubernetes framework is responsible for provisioning the 5G core network by deploying Docker images sourced from the OpenAirInterface (OAI) repository. The core network Docker container interfaces with the shared RAN infrastructure via the N3 (user plane) and N2 (control plane) interfaces. The gNB maintains and dynamically updates the mapping between network slices and the core network. During deployment, the User Equipment (UE) transmits the slice ID, allowing the gNB to appropriately route UE traffic to the corresponding core network functions. The RAN Slice Orchestrator (RSO) initiates and schedules 5G RAN slices.

gNB is managed and controlled via the FlexRAN protocol [42]. There is a unique scheduler for each slice to schedule its UEs. We schedule eMBB using a proportional fair approach, uRLLC using delay-based scheduling, and mMTC using round-robin. The scheduling handler is responsible for

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

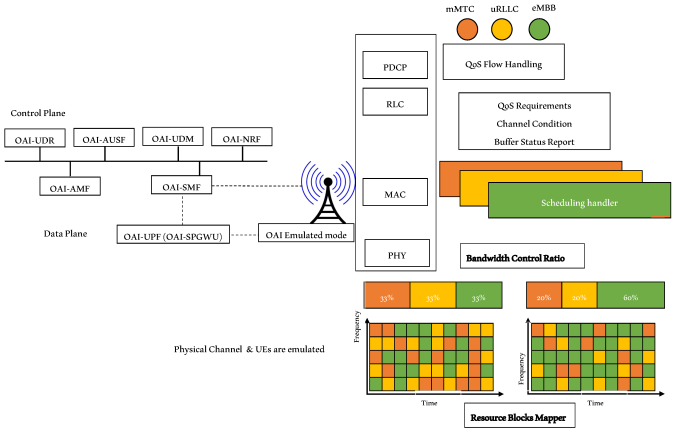


Fig. 3. TestBed for network slicing based on OAI-5G.

allocating virtual Resource Blocks (vRBs) to UEs according to the type of slice. To determine the bandwidth control ratio (BCR), each scheduler combines the information of QoS requirements, channel quality indicators (CQIs), channel conditions, and buffer status reports from upper and lower layers. BCR is a policy that enforces slice requirements based on slice traffic demands, which can vary statically or dynamically. The ratio [33%, 33%, 33%] in Figure 3 indicates that [mMTC, uRLLC, and eMBB] received 33% of the physical resource blocks upon the slice request. To ensure fairness across slices with different priority levels, our data collection process captures diverse traffic patterns under both typical and contention-heavy conditions. While high-priority slices such as uRLLC require strict latency and reliability guarantees, we ensure that lower-priority slices (e.g., mMTC) are also consistently represented during data generation and model training. We simulate overlapping high-demand scenarios where all slices experience simultaneous traffic surges. This enables the learning agent to observe real-world resource contention and to learn allocation strategies that consider the needs of all slices. Importantly, resource demand, latency, and throughput metrics are logged uniformly across slices, preventing the model from overfitting to high-frequency or high-priority traffic. In addition, we adopt a fairness-preserving reward function during training, which penalizes QoS violations and rewards consistent performance across all slices. During deployment, we maintain a baseline resource floor (e.g., equal initial BCR ratios), ensuring that no slice is starved, even during dynamic reallocation triggered by traffic surges.

The system operates over a 200 MHz carrier bandwidth, corresponding to 100 physical resource blocks (PRBs) per 1 ms Transmission Time Interval (TTI), which is a typical configuration in 5G NR mid-band deployments. This granularity allows for highly responsive scheduling, particularly important for uRLLC slices where sub-millisecond reaction times can be critical. Although our testbed does not explicitly simulate UE mobility or detailed channel effects like fading and shadowing, these are approximated through FlexRAN's built-in PHY/MAC abstractions, which include CQI-based adaptation, RLC retransmissions, and random channel variations.

TABLE I
NETWORK SCENARIO SETTINGS

Parameter	Slice 1	Slice 2	Slice 3
Type of Service	uRLLC (Emergency)	eMBB	mMTC
Number of Users	20	80	50
Request Interval (ms)	5	10	50
Throughput Requirement (Mbps)	10	50	1
Latency Requirement (ms)	10	50	100
Resource Allocation Priority	High	Medium	Low

Inter-slice interference and network congestion are captured through dynamic PRB contention and injected bursty traffic. Together, these mechanisms create realistic network slice conditions, enabling us to focus on analyzing slice-level behaviors.

To emulate realistic traffic profiles, we inject high-burst traffic with average packet sizes of 1400 bytes (eMBB), 500 bytes (uRLLC), and 100 bytes (mMTC), reflecting common application-layer behaviors in video streaming, industrial automation, and IoT. Our primary focus is on slice-level allocation; lower-layer UE properties such as mobility, channel fading, and user-level resource mapping are managed by the default configurations of the FlexRAN-based testbed. This allows us to isolate and analyze the impact of slice-level intelligence without interference from underlying PHY/MAC variability. This is particularly critical for delay-sensitive services such as emergency response (Slice 1), where timely adaptation is essential. In our simulation, this mechanism is implicitly triggered by the high-frequency request pattern (every 5 ms) and bursty traffic conditions of the emergency slice. In addition to periodic updates, the testbed simulates event-triggered conditions via 5 ms high-frequency traffic pattern of slice 1. To explicitly examine this behavior, we isolate a time window in Section VII where a traffic spike prompts reallocation, revealing the adaptive role of our confidence-aware mechanism. All experiments were conducted using the 5G testbed developed for this study, ensuring a realistic simulation of network conditions.

VI. EXPERIMENTAL SETTINGS

A. Data Collection

We build on the testbed from Section V by configuring a 5G RAN environment with 100 RBs per 200 ms interval. Based on the 3GPP specifications, the network hosts three slices with distinct QoS requirements, summarized in Table I.

- **Slice 1 (Emergency Services)** represents delay-sensitive and highly dynamic applications like emergency response or industrial control. It serves 20 users generating requests every 5 ms, leading to bursty, high-frequency traffic. To ensure rapid responses, the slice enforces a 10 Mbps throughput floor and 10 ms latency cap, making it the primary driver for our *event-driven reallocation* mechanism (see Section III-A).
- **Slice 2 (eMBB)** models enhanced Mobile Broadband scenarios (e.g., video streaming). It supports 80 users, each sending requests every 10 ms. The slice requires 50 Mbps aggregate throughput and tolerates latencies of up to 50 ms.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MODEL CONFIGURATION AND HYPERPARAMETERS FOR PRENS

Parameter	Value	Search Space
Bi-LSTM Predictor (PreDP)		
Input Sequence Length	5 steps (500 ms)	$[200\text{ ms}, \cdots, 800\text{ ms}]$
Hidden Units (per direction)	64	$\{32, 64, 128\}$
Attention Mechanism	Additive + Softmax	Fixed
Learning Rate (α)	0.001	$[5 \times 10^{-4}, \cdots, 5 \times 10^{-1}]$
Optimizer	Adam	Fixed
Loss Function	MSE	Fixed
Batch Size	32	$\{16, 32, 64\}$
Training Epochs	600	$[200, \cdots, 1000]$
DDQN Agent		
Network Architecture	[128, 64] (ReLU)	Fixed
Discount Factor (γ)	0.95	$\{0.9, 0.95, 0.99\}$
Exploration (ϵ) Init $\rightarrow$ Final	$1.0 \rightarrow 0.05$	Fixed
Decay Steps	10,000	$\{5k, 10k, 20k\}$
Replay Buffer Size	20,000	$\{10k, 20k, 50k\}$
Batch Size	64	$\{32, 64, 128\}$
Target Network Update	Every 200 steps	$\{100, 200, 500\}$
Reward Normalization	$[-1, 1]$	Fixed
Reward Weights (α, β)	0.6, 0.4	$[0.1, \cdots, 0.9]$
Fairness Penalty Weight (γ)	0.3 (Tuned)	$[0.1, \cdots, 1.0]$
Optimizer	Adam	Fixed
Learning Rate	0.001	$[5 \times 10^{-4}, \cdots, 5 \times 10^{-2}]$

- **Slice 3 (mMTC)** targets massive IoT deployments. It accommodates 50 users with request intervals of 50 ms, a minimal 1 Mbps throughput requirement, and a high 100 ms latency tolerance. This slice is assigned the lowest priority.

At each allocation step, the base station receives aggregated slice-level demands and executes resource assignment accordingly. The PreDP module forecasts the next 200 ms [34], [43] demand based on the past 500 ms of slice activity (see Table II). Two data streams are collected for each decision point: (i) *historical traffic data*: a 500 ms sequence of past slice demands and resource usage; and (ii) *real-time state data*: up-to-date metrics such as available RBs, user activity, and QoS performance indicators.

Section VI-A now undergoes smoothing and interpolation before feeding the Bi-LSTM predictor: First, we employ *Unit-Time Smoothing*. Following [16], each user request R_o^m in slice o is evenly distributed across its latency window ξ_o, resulting in a normalized rate: $UTR_o^m = \frac{R_o^m}{\xi_o}$. For example, if a request of 10 RBs arrives at time t with a 10 ms latency requirement, it is decomposed into 10 unit-time intervals, each assigned 1 RB at times $\{t, t{+}1, \ldots, t{+}9\}$. This smoothing approach effectively addresses the majority of short-gap scenarios encountered in network traffic and improves input consistency for time-series modeling. Next, we apply linear interpolation for gaps up to two steps, median filtering (window size = 3) to suppress outliers, and min–max normalization to scale traffic values into $[0, 1]$ per slice.

B. Prediction Evaluation Metrics

To assess the forecasting performance of PreDP, we use the following metrics jointly evaluating prediction accuracy, scale, and variance explanation:

- **Root Mean Squared Error (RMSE)**:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- **Mean Absolute Error (MAE)**: $\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
- **Coefficient of Determination (R^2)**:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

C. Replay Buffer Management and Generalization

We use a FIFO replay buffer with a fixed size of 20,000 transitions to store training experiences. Transitions are sampled uniformly to avoid slice bias and to expose the agent to varied conditions, including traffic bursts, priority conflicts, and SLA violations. We experimented with PER but observed marginal improvements at the cost of training instability. In highly dynamic network environments, PER tends to amplify rare but noisy transitions, which can skew the agent's policy toward overfitting or instability. Instead, we apply mini-batch sampling with shuffling during training. This helps break sequential correlations and ensures the agent generalizes across temporal patterns rather than memorizing local behaviors. In each training run, we vary the traffic burst sizes, inter-arrival intervals, and session start times, ensuring that the replay buffer collects experiences from a broad set of traffic behaviors and load conditions. To reduce overfitting, the model is periodically evaluated on unseen validation traces containing both bursty and regular demand patterns.

VII. RESULTS

This section presents the empirical evaluation of PreNS, focusing on prediction accuracy, resource allocation efficiency, and responsiveness under dynamic 5G RAN conditions. All models are implemented in PyTorch and trained on an Nvidia RTX 4090 GPU with an Intel Core i9-13900K CPU, following the setup described in Section VI. We select hyperparameters for both PreDP and the DDQN agent via grid search (Table II). The predictor is optimized using mean squared error (MSE), while the DDQN relies on a standard temporal-difference loss. To ensure statistical robustness, each experiment is repeated five times, and we report the average performance. In the subsections that follow, we benchmark PreNS against state-of-the-art baselines, evaluating forecasting performance, QoS satisfaction, and overall system utility.

A. Prediction Performance

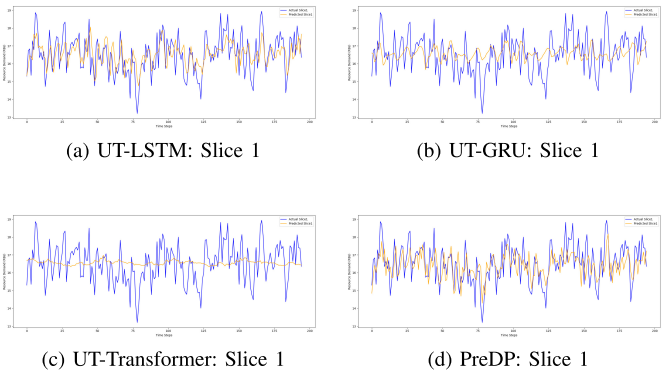
To evaluate the effectiveness of the PreDP model in forecasting slice-level resource demands, we benchmark it against four established baselines:

1) **UT-LSTM** [16]: A two-layer unidirectional LSTM that models temporal dependencies using forward-sequential processing.
2) **UT-GRU** [15]: A two-layer Gated Recurrent Unit network offering a lightweight alternative to LSTM while retaining long-range memory capabilities.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMBINED PERFORMANCE AND EFFICIENCY OF DIFFERENT MODELS

Model	# Layers	# Params	Train Time (min)	Inference Time (ms)	GPU memory use(MB)	Peak GPU memory (MB)	RMSE	MAE	R^2
UT-LSTM [16]	2	51,139	7.6	0.977	0.20	26.57	0.8677	0.6261	0.4523
UT-GRU [15]	2	38,403	4.6	0.550	0.15	20.39	0.8234	0.5841	0.5183
UT-Transformer [19]	2	283,067	31.2	0.603	1.13	54.07	1.0922	0.8243	0.2813
UT-ARIMA [10]	-	-	-	2.015	-	-	2.2645	1.5329	0.1263
UT-GRU [15]	1	13,443	**2.1**	**0.183**	**0.11**	**18.07**	1.0143	0.7214	0.4183
PreDP	1	**35,844**	10.6	0.467	0.14	31.09	**0.6977**	**0.5141**	**0.5762**



(a) UT-LSTM: Slice 1 (b) UT-GRU: Slice 1

(c) UT-Transformer: Slice 1 (d) PreDP: Slice 1

Fig. 4. Comparison of different predictive methods with event-driven slice (Slice 1). The blue line represents actual demand, while the orange line shows the predicted demand.

3) **UT-Transformer** [19]: A self-attention-based model effective for learning long-range correlations, but typically requiring large-scale data and hyperparameter tuning.
4) **UT-ARIMA** [10]: A classical statistical model suitable for linear trends, though limited in handling the non-linear characteristics of wireless traffic.

1) Real-Time Suitability: Table III presents a comprehensive comparison of prediction accuracy and system efficiency. PreDP achieves the best trade-off among all models, with the lowest RMSE (0.6977), highest R^2 (0.5762), and a low inference latency of 0.467 ms, well within the 5 ms allocation cycle used for event-driven slices. This demonstrates its suitability for real-time deployment in live RAN environments. In contrast, while the GRU model yields the fastest inference (0.183 ms), it suffers from reduced accuracy. The ARIMA baseline underperforms across all metrics, reinforcing its inadequacy for modeling non-linear and bursty traffic.

2) Impact of Attention and Recurrence: Although Transformer-based models excel in large-scale sequence modeling, their performance degrades on smaller, noisy time-series data without extensive tuning. This is reflected in both the quantitative metrics and the visual results in Figure 4. In comparison, PreDP consistently aligns more closely with the actual demand curves, particularly under dynamic traffic fluctuations—highlighting the advantage of combining recurrent structures with attention mechanisms.

3) Robustness to Bursty Traffic: To assess resilience under extreme traffic conditions, we focus on Slice 1 (Figure 4), which emulates uRLLC-like behavior with 5 ms event-driven request intervals. While PreDP delivers the most accurate forecasts on this slice, all models, including ours, still exhibit decreased accuracy when confronted with sudden spikes. These abrupt surges underscore why even a high-performing predictor cannot be trusted exclusively in highly volatile scenarios.

To address this, our framework employs a *confidence-aware state aggregator*, which gauges the reliability of forecasts in real time and dynamically adjusts how they are weighted against live measurements. When PreDP's predictions are consistent with recent observations, the system leans more heavily on forecasts to allocate resources proactively. Conversely, if the model begins to diverge (as often happens with bursty traffic), the aggregator pivots toward immediate feedback from the network to prevent misallocation. As a result, PreNS remains stable in the presence of frequent demand spikes, confirming that a hybrid of predictive foresight and reactive adaptability is crucial for burst-tolerant, resource forecasting.

B. Allocation System Performance

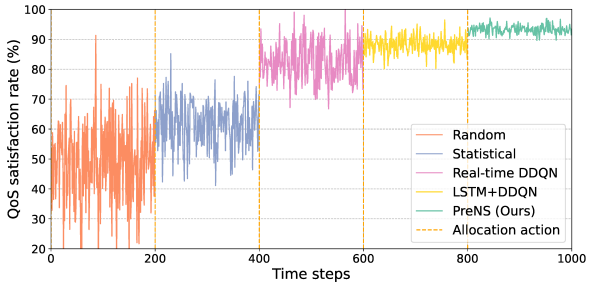
We evaluate PreNS over 1,000 time steps (five intervals) using three key metrics: i) *QoS Satisfaction Rate*, the proportion of users meeting latency/throughput targets; ii) *Bandwidth Utilization*, the effective use of available bandwidth without under- or over-provisioning; and iii) *Latency*, the average end-to-end delay for time-sensitive services. PreNS is compared against four baselines:

- **Random Allocation**: A naive lower bound with no demand or QoS awareness.
- **Statistical Allocation**: Uses historical average demand per slice; simple but assumes traffic stationarity and lacks adaptability.
- **Real-Time DDQN** [34]: Allocates based on current state only, responsive but unable to anticipate demand surges.
- **LSTM+DDQN** [35]: Combines LSTM forecasting with DDQN allocation, but lacks dynamic fusion of predicted and real-time states.

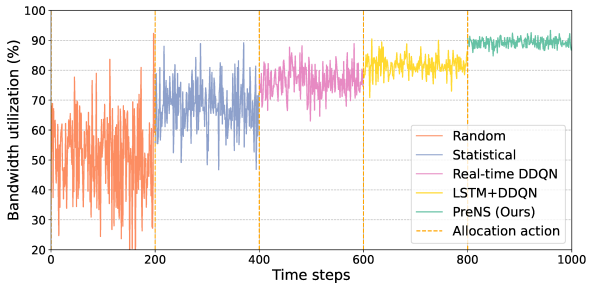
1) QoS Satisfaction Rate: We evaluate the effectiveness of each resource allocation strategy, with decisions made every 200 ms in alignment with the RAN scheduling cycle. As shown in Figure 5a, the PreNS framework achieves a QoS Satisfaction Rate of 93.4%, outperforming all baseline methods with lower variance and greater stability.

The Random allocation method yields the lowest satisfaction rate, showing that unguided resource distribution fails to meet user needs. Statistical allocation improves this to 62.3% using historical averages but cannot adapt to real-time fluctuations. Real-Time DDQN performs better by reacting to

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.



(a) Average QoS satisfaction rate of different methods



(b) Average bandwidth utilization of different methods

Fig. 5. Comparison of average QoS satisfaction and bandwidth utilization across different methods.

current conditions, yet it lacks foresight and suffers under abrupt demand changes. LSTM+DDQN improves further, benefiting from foresight but limited by the absence of joint state aggregation. In contrast, PreNS combines accurate demand prediction with confidence-weighted state fusion, enabling consistent high performance. The observed **14.1%** improvement over Real-Time DDQN and **5.2%** gain over LSTM+DDQN highlights its superior adaptability to dynamic and heterogeneous QoS demands.

2) Bandwidth Utilization: As illustrated in Figure 5b, PreNS achieves the highest bandwidth utilization at 89.2%, demonstrating its efficiency in allocating resources to match both real-time and predicted demands. This balanced approach minimizes both under-provisioning (which can cause QoS violations) and over-provisioning (which wastes capacity). In comparison, Random allocation achieves just 52.1%, reflecting the inefficiency of non-strategic distribution. Statistical allocation improves utilization to 68.0% by leveraging historical data but struggles to respond to real-time variability. Real-Time DDQN (76.3%) benefits from immediate feedback but lacks foresight, while LSTM+DDQN (81.5%) gains from prediction but does not adapt dynamically to inaccuracies or unexpected events. The superior utilization rate of PreNS confirms that confidence-aware fusion of prediction and real-time state enables smarter and more responsive resource use, crucial for high-density, multi-service RAN environments.

3) Latency Analysis: Figure 6 shows the latency distributions for the three slices: Slice 1 (Emergency), Slice 2 (eMBB), and Slice 3 (mMTC). PreNS achieves consistently lower latencies: 6.5 ms, 35.6 ms, and 69 ms; representing improvements of 18.2%, 24.7%, and 16.5% over the best baselines, respectively. This confirms PreNS's ability to support diverse QoS classes simultaneously. For latency-critical services like Slice 1 (5 ms interval), proactive allocation and rapid response to bursts are key to maintaining SLA guarantees.

TABLE IV
ABLATION RESULTS WITH CONFIDENCE-AWARE FUSION VARIANTS

Configuration	QoS (%)	BW Util. (%)	Latency (ms)	Infer. (ms)
Without Attention Mechanism	91.1	85.6	42.5	1.015
DQN instead of DDQN	89.3	83.8	44.7	0.998
Without Aggregated State	87.9	81.5	47.2	0.642
$C_r = 1$ (Real-Time Only)	81.6	76.3	49.8	0.571
$C_p = 1$ (Prediction Only)	85.1	79.8	47.6	1.548
Static Fusion (0.5/0.5)	86.8	80.5	46.2	1.003
Full Model (PreNS)	**93.4**	**89.2**	**37.0**	1.097

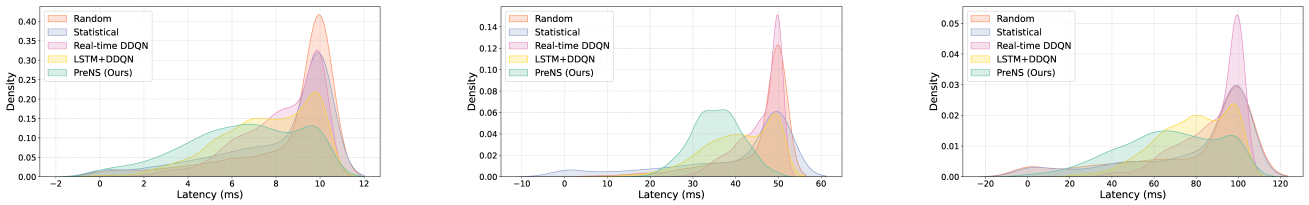
These gains stem from PreNS's dynamic balance of predictive foresight and real-time adaptability. By assigning confidence weights to forecasts and live measurements, the framework reallocates resources in time to avoid queuing delays (Fig. 6a). In contrast, Real-Time DDQN reacts only after congestion occurs, while LSTM+DDQN relies solely on forecasts without reliability checks, leading to over- or under-provisioning. As a result, PreNS delivers higher QoS satisfaction, improved utilization, and tighter delay bounds across all slice types, demonstrating the effectiveness of combining proactive forecasting with adaptive correction in dynamic RAN environments.

4) Inference Time: To evaluate the efficiency of PreNS in real-time scenarios, we measured the inference times of multiple approaches, including Random Allocation, Statistical Allocation, real-time DDQN, and LSTM+DDQN. Random and Statistical Allocation both exhibit near-zero inference time due to their straightforward strategies, whereas real-time DDQN requires 0.571 ms and LSTM+DDQN 1.548 ms. In comparison, PreNS takes 1.097 ms—primarily due to predictive modeling and aggregated state processing—yet achieves markedly superior QoS results. Because resource allocation occurs every 100 ms, this extra 1 ms overhead has a negligible impact relative to the significant performance gains, as also shown in Table IV.

5) Latency Reduction During Traffic Spikes: A key advantage of PreNS is its ability to reduce latency during sudden traffic surges, particularly for delay-sensitive slices such as Slice 1. Unlike Real-Time DDQN, which reacts only after congestion is observed, PreNS leverages Bi-LSTM–based predictions to anticipate demand increases. Through its confidence-aware aggregation mechanism, it adaptively balances predictive and real-time inputs according to their recent reliability. This enables proactive allocation when predictions are trustworthy, while reverting to real-time data when prediction uncertainty is high. By contrast, the LSTM+DDQN baseline lacks this adaptive capability and often relies on raw predictions without accounting for confidence, leading to over- or under-provisioning during bursty periods.

As shown in Figure 6 and Table IV, PreNS achieves substantially lower average and tail latencies under high load, up to 30% reduction in average delay for Slice 1 compared to Real-Time DDQN, and maintains more stable QoS adherence than LSTM+DDQN. These results demonstrate the effectiveness of

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.



(a) Slice 1 (uRLLC Emergency) (b) Slice 2 (eMBB) (c) Slice 3 (mMTC)

Fig. 6. Latency distribution across different network slices under the proposed slicing framework.

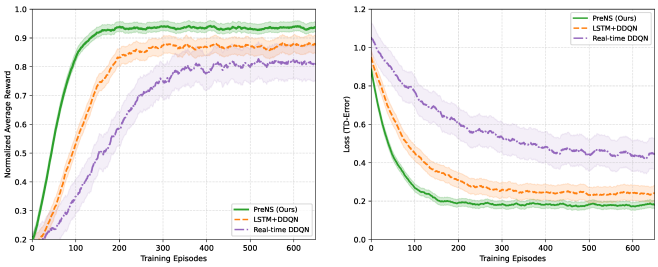
combining proactive forecasting with confidence-aware fusion for dynamic slice management.

C. Ablation Study

To assess the contribution of each component in PreNS and motivate the overall design, we perform an ablation study by systematically removing or modifying modules and observing the impact on performance. The configurations evaluated are:

- **Full Model (PreNS)**: The complete framework with the PreDP predictor, DDQN-based allocator, and confidence-aware fusion of predicted and real-time states.
- **Without Attention**: A Bi-LSTM predictor without the attention mechanism, used to measure the benefit of attention on forecasting accuracy.
- **DQN instead of DDQN**: Replaces the DDQN with a standard DQN to evaluate the effect of overestimation bias on allocation stability.
- **Without Aggregated State**: The allocator uses only real-time observations, omitting predicted demand, to assess the value of combining forecasts with live data.
- $C_r = 1$ **(Real-Time Only)**: An artificial baseline where the allocator relies exclusively on real-time inputs, illustrating performance without prediction.
- $C_p = 1$ **(Prediction Only)**: An artificial baseline where the allocator relies solely on predicted demand, illustrating performance without real-time adaptation.
- **Static Fusion** ($C_p = C_r = 0.5$): A fixed-weight baseline that combines prediction and real-time inputs equally, used to contrast with adaptive fusion.

Table IV highlights the critical contributions of each PreNS component. Removing the attention mechanism reduces the QoS satisfaction rate from 93.4% to 91.1%, bandwidth efficiency from 89.2% to 85.6%, and increases latency from 37.0 ms to 42.5 ms, showing its role in enhancing prediction accuracy by focusing on relevant temporal features. Replacing DDQN with DQN further lowers QoS satisfaction to 89.3% and bandwidth efficiency to 83.8%. This degradation stems from DQN's tendency to overestimate Q-values, which can lead to suboptimal actions in a highly dynamic and stochastic environment. DDQN addresses this issue by decoupling action selection and evaluation, resulting in more stable learning and better overall performance in our hybrid allocation framework. Excluding the aggregated state, where the agent relies solely on real-time data, leads to a significant drop in performance, emphasizing the importance of combining predictions with real-time observations for proactive resource management.



(a) Average reward (b) TD-error dynamics

Fig. 7. Convergence analysis of PreNS compared with baseline methods. (a) Average reward per training episode (shaded regions denote the standard deviation), illustrating the convergence behavior of different methods. (b) Training loss (TD-error) evolution over time, reflecting the stability of temporal-difference updates during learning.

We also analyze the inference time of each configuration to evaluate its feasibility in real-time scenarios. While the full PreNS model has the highest inference time at 1.097 ms, it remains well within the acceptable threshold for 100 ms-level resource allocation cycles. Moreover, compared to the LSTM+DDQN baseline (1.548 ms), PreNS achieves a better trade-off between latency, efficiency, and inference overhead.

The results confirm the importance of each component in PreNS. The attention mechanism enhances prediction accuracy for proactive allocation. DDQN improves learning stability and resource allocation by mitigating overestimation bias. The aggregated state enables both proactive and reactive adjustments, addressing the dynamic demands of B5G RAN environments. This validates the design of PreNS, where every component is essential for optimal performance in B5G RAN slicing.

D. Learning Dynamics and Convergence Analysis

To further examine the impact of adaptive fusion on learning stability and convergence, we analyze the training dynamics of PreNS and the baseline methods in terms of reward evolution and TD-error behavior. Here, the TD-error corresponds to the temporal-difference loss used to update the Q-network, whose magnitude reflects the stability of value estimation during training [44].

Fig. 7(a) shows the normalized average reward as a function of training episodes. For clarity, the reward curves are smoothed using a moving average over 20 episodes. PreNS converges faster and achieves a higher steady-state reward compared with both LSTM+DDQN and real-time DDQN.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

In addition, the learning curve of PreNS is smoother and exhibits significantly lower variance (indicated by the narrower shaded region) during training, indicating more stable policy updates and improved convergence behavior in practice. Fig. 7(b) presents the evolution of the TD-error over time. PreNS consistently maintains lower TD-error values and demonstrates faster decay than the baseline methods. This behavior suggests that the Q-value updates in PreNS are more stable and less sensitive to abrupt state variations, thereby reducing oscillatory updates during learning.

These observations are consistent with the design of the proposed confidence-aware adaptive fusion. By dynamically weighting predicted and real-time signals according to their reliability, PreNS mitigates the impact of inaccurate predictions and volatile traffic on the state representation, effectively reducing the impact of state noise. This results in smoother TD-error dynamics and improved convergence stability.

E. Discussions

1) Impact of Confidence-Aware Fusion: We performed additional ablation experiments to evaluate the contribution of the confidence-aware fusion module (Table IV). Adaptive fusion in PreNS achieves the best performance. In contrast, using only real-time feedback (C_r) degrades satisfaction by 11.8% and increases latency by 34.6%; prediction-only fusion (C_p) lowers satisfaction by 8.3% with a 28.6% latency increase. A static fusion strategy (0.5/0.5) reduces satisfaction by 6.6% and raises latency by 24.9%. These results confirm that eliminating adaptive balancing significantly harms performance, underscoring the importance of confidence-aware fusion for robust slice management.

2) Impact of Network Dynamics: While our primary experiments focus on benchmarking PreNS against baseline models, the evaluation setup implicitly includes challenging network conditions. In particular, the emergency (uRLLC) slice introduces high-burst traffic with 5 ms request intervals, testing PreNS under extreme demand volatility. The framework demonstrates strong performance in this setting, maintaining accurate predictions and high QoS satisfaction, as shown in Figures 4–6. However, user mobility and its effect on resource allocation were abstracted in our current testbed, which adopts default lower-layer configurations for mobility and scheduling (see Section V). As a result, the impact of heterogeneous mobility speeds on prediction and allocation is not explicitly modeled in this study.

3) Scalability and Real-Time Efficiency: To evaluate the real-world applicability of PreNS in large-scale 5G and B5G deployments, we conducted both analytical and empirical assessments of its computational efficiency under increasing system load. Two primary scalability dimensions were explored: (i) *user-level scalability*, and (ii) *slice-level scalability*. Table V summarizes these runtime characteristics.

At the user level, PreNS operates at the slice granularity, meaning that the number of users within a slice affects the total demand but not the model's input dimensionality. As a result, inference time for the LSTM-based predictor remains effectively constant at 0.467 ms across 100 to 1000 users. The DDQN inference time increases marginally from 0.63 ms to 0.713 ms due to larger state-action interactions, yet remains comfortably within real-time constraints. This decoupling ensures robust performance even as user populations scale.

TABLE V
SCALABILITY AND EFFICIENCY OF PRENS UNDER VARYING USER AND SLICE COUNTS

Scenario	Inference Time (ms)		Training Time	
	PreDP	DDQN	PreDP (min)	DDQN (hr)
User-Level Scalability (Slice Count = 3)				
100	0.467	0.630	10.61	0.49
200	0.466	0.649	10.60	0.56
500	0.467	0.684	10.62	0.63
1000	0.466	0.713	10.61	0.70
Slice-Level Scalability (User Count = 500)				
3	0.467	0.684	10.62	0.63
4	0.471	0.734	10.68	0.74
5	0.468	0.725	10.67	1.24
10	0.467	0.750	10.98	2.37
15	0.494	0.795	11.01	3.45

At the slice level, where the input size grows linearly with the number of slices, both components of PreNS maintain high computational efficiency. The predictor's inference latency remains near-constant, while the DDQN agent's inference time gradually rises from 0.684 ms (3 slices) to 0.795 ms (15 slices). Training time for the LSTM predictor remains stable, demonstrating model robustness to input dimensionality. In contrast, DDQN training time increases, reflecting the nonlinear growth of the action space and associated exploration requirements. Notably, the complete decision pipeline, including forecasting and real-time resource allocation, executes in under 1.1 ms, satisfying the ultra-low latency constraints of uRLLC services. These results confirm that PreNS is scalable and efficient for real-time deployment in dense, multi-slice 5G/B5G network environments.

VIII. CONCLUSION

In this paper, we presented PreNS, a framework that integrates predictive modeling with real-time decision-making to optimize resource allocation in 5G RAN slicing. By combining a Bi-LSTM with attention for accurate demand forecasting and a DDQN for adaptive resource allocation, PreNS balances proactive and reactive strategies through an aggregated state that fuses predicted and real-time data. Experimental results show that PreNS significantly outperforms traditional methods in prediction accuracy, resource utilization, QoS satisfaction, and latency. The ablation study confirms the importance of each component, validating our approach. PreNS effectively addresses the challenges of dynamic network conditions, offering a scalable and efficient solution for 5G networks.

ACKNOWLEDGMENT

Any opinions, findings and conclusion or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore.

REFERENCES

[1] W. Rafique, J. Rani Barai, A. O. Fapojuwo, and D. Krishnamurthy, "A survey on beyond 5G network slicing for smart cities applications," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 1, pp. 595–628, Feb. 2025.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.

[2] N. Alliance, "5G white paper," *Next Generat. Mobile Netw.*, vol. 1, pp. 1–124, Feb. 2015.

[3] A. A. Barakabitze and R. Walshe, "SDN and NFV for QoE-driven multimedia services delivery: The road towards 6G and beyond networks," *Comput. Netw.*, vol. 214, Sep. 2022, Art. no. 109133.

[4] W. Hamdi, C. Ksouri, H. Bulut, and M. Mosbah, "Network slicing-based learning techniques for IoV in 5G and beyond networks," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 3, pp. 1989–2047, 3rd Quart., 2024.

[5] M. Z. Chowdhury, M. Shahjalal, S. Ahmed, and Y. M. Jang, "G wireless communication systems: Applications, requirements, technologies, challenges, and research directions," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 957–975, 2020.

[6] Z. Zhao, M. Peng, Z. Ding, W. Wang, and H. V. Poor, "Cluster content caching: An energy-efficient approach to improve quality of service in cloud radio access networks," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 5, pp. 1207–1221, May 2016.

[7] C. Sun, C. She, C. Yang, T. Q. S. Quek, Y. Li, and B. Vucetic, "Optimizing resource allocation in the short blocklength regime for ultra-reliable and low-latency communications," *IEEE Trans. Wireless Commun.*, vol. 18, no. 1, pp. 402–415, Jan. 2019.

[8] Y. Azimi, S. Yousefi, H. Kalbkhani, and T. Kunz, "Energy-efficient deep reinforcement learning assisted resource allocation for 5G-RAN slicing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 856–871, Jan. 2022.

[9] F. Mason, G. Nencioni, and A. Zanella, "Using distributed reinforcement learning for resource orchestration in a network slicing scenario," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 88–102, Feb. 2023.

[10] N. Bui and J. Widmer, "Data-driven evaluation of anticipatory networking in LTE networks," *IEEE Trans. Mobile Comput.*, vol. 17, no. 10, pp. 2252–2265, Oct. 2018.

[11] V. Sciancalepore, K. Samdanis, X. Costa-Perez, D. Bega, M. Gramaglia, and A. Banchs, "Mobile traffic forecasting for maximizing 5G network slicing resource utilization," in *Proc. IEEE Conf. Comput. Commun.*, May 2017, pp. 1–9.

[12] D. Bega, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Perez, "DeepCog: Optimizing resource provisioning in network slicing with AI-based capacity forecasting," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 361–376, Feb. 2020.

[13] D. Bega, M. Gramaglia, A. Garcia-Saavedra, M. Fiore, A. Banchs, and X. Costa-Perez, "Network slicing meets artificial intelligence: An AI-based framework for slice management," *IEEE Commun. Mag.*, vol. 58, no. 6, pp. 32–38, Jun. 2020.

[14] E. H. Bouzidi, A. Outtagarts, A. Hebbar, R. Langar, and R. Boutaba, "Online based learning for predictive end-to-end network slicing in 5G networks," in *Proc. ICC - IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–7.

[15] H. Chergui and C. Verikoukis, "Big data for 5G intelligent network slicing management," *IEEE Netw.*, vol. 34, no. 4, pp. 56–61, Jul. 2020.

[16] W. Binghui, N. V. Abhishek, P. C. Amogh, and M. Gurusamy, "NPRA: A novel predictive resource allocation mechanism for next generation network slicing," in *Proc. IEEE 20th Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2023, pp. 716–721.

[17] K. S. H. Ong, W. Wang, D. Niyato, and T. Friedrichs, "Deep-reinforcement-learning-based predictive maintenance model for effective resource management in industrial IoT," *IEEE Internet Things J.*, vol. 9, no. 7, pp. 5173–5188, Apr. 2022.

[18] H. Zheng, F. Lin, X. Feng, and Y. Chen, "A hybrid deep learning model with attention-based conv-LSTM networks for short-term traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 6910–6920, Nov. 2021.

[19] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2017, pp. 5598–6008.

[20] C. Marquez, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Perez, "Resource sharing efficiency in network slicing," *IEEE Trans. Netw. Service Manage.*, vol. 16, no. 3, pp. 909–923, Sep. 2019.

[21] C.-W. Huang, C.-T. Chiang, and Q. Li, "A study of deep learning networks on mobile traffic forecasting," in *Proc. IEEE 28th Annu. Int. Symp. Pers., Indoor, Mobile Radio Commun. (PIMRC)*, Oct. 2017, pp. 1–6.

[22] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space Odyssey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2222–2232, Oct. 2017.

[23] R. Li, C. Wang, Z. Zhao, R. Guo, and H. Zhang, "The LSTM-based advantage actor-critic learning for resource management in network slicing with user mobility," *IEEE Commun. Lett.*, vol. 24, no. 9, pp. 2005–2009, Sep. 2020.

[24] H. P. Phyu, D. Naboulsi, and R. Stanica, "Machine learning in network slicing—A survey," *IEEE Access*, vol. 11, pp. 39123–39153, 2023.

[25] B. Wu, D. Chen, N. V. Abhishek, and M. Gurusamy, "D3T: Double deep Q-network decision transformer for service function chain placement," in *Proc. IEEE 24th Int. Conf. High Perform. Switching Routing (HPSR)*, Jun. 2023, pp. 167–172.

[26] C. Qi, Y. Hua, R. Li, Z. Zhao, and H. Zhang, "Deep reinforcement learning with discrete normalized advantage functions for resource management in network slicing," *IEEE Commun. Lett.*, vol. 23, no. 8, pp. 1337–1341, Aug. 2019.

[27] S. Meng et al., "RAN slice strategy based on deep reinforcement learning for smart grid," in *Proc. Comput., Commun. IoT Appl. (ComComAp)*, Oct. 2019, pp. 6–11.

[28] C. Zhang, M. Dong, and K. Ota, "Vehicular multi-slice optimization in 5G: Dynamic preference policy using reinforcement learning," in *Proc. GLOBECOM - IEEE Global Commun. Conf.*, Dec. 2020, pp. 1–6.

[29] D. Shome and A. Kudeshia, "Deep Q-learning for 5G network slicing with diverse resource stipulations and dynamic data traffic," in *Proc. Int. Conf. Artif. Intell. Inf. Commun. (ICAIIC)*, Apr. 2021, pp. 134–139.

[30] J. Mei, X. Wang, K. Zheng, G. Boudreau, A. B. Sediq, and H. Abou-Zeid, "Intelligent radio access network slicing for service provisioning in 6G: A hierarchical deep reinforcement learning approach," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 6063–6078, Sep. 2021.

[31] Q. Liu, T. Han, N. Zhang, and Y. Wang, "DeepSlicing: Deep reinforcement learning assisted resource allocation for network slicing," in *Proc. GLOBECOM - IEEE Global Commun. Conf.*, Dec. 2020, pp. 1–6.

[32] M. Yan, G. Feng, J. Zhou, Y. Sun, and Y.-C. Liang, "Intelligent resource scheduling for 5G radio access network slicing," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, pp. 7691–7703, Aug. 2019.

[33] K. Yu, H. Zhou, B. Qian, Z. Tang, and X. Shen, "A reinforcement learning aided decoupled RAN slicing framework for cellular V2X," in *Proc. IEEE Global Commun. Conf.*, Dec. 2020, pp. 1–6.

[34] T. Li, X. Zhu, and X. Liu, "An end-to-end network slicing algorithm based on deep Q-learning for 5G network," *IEEE Access*, vol. 8, pp. 122229–122240, 2020.

[35] Y. Cui, X. Huang, P. He, D. Wu, and R. Wang, "A two-timescale resource allocation scheme in vehicular network slicing," in *Proc. IEEE 93rd Veh. Technol. Conf. (VTC-Spring)*, Apr. 2021, pp. 1–5.

[36] K. Tian, Y. Wang, D. Pan, and D. Yuan, "DRL-based dynamic resource configuration and optimization for B5G network slicing," *IEEE Access*, vol. 12, pp. 120864–120876, 2024.

[37] A. Gharehgoli, A. Nouruzi, N. Mokari, P. Azmi, M. R. Javan, and E. A. Jorswieck, "AI-based resource allocation in end-to-end network slicing under demand and CSI uncertainties," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 3630–3651, Sep. 2023.

[38] D. López-Pérez, A. Ladányi, A. Jüttner, H. Rivano, and J. Zhang, "Optimization method for the joint allocation of modulation schemes," in *Proc. IEEE INFOCOM*, Apr. 2011, pp. 111–115.

[39] *OpenAirInterface5G*. Accessed: Dec. 27, 2024. [Online]. Available: https://gitlab.eurecom.fr/oai/openairinterface5g

[40] *Kubernetes*. Accessed: Dec. 27, 2024. [Online]. Available: https://kubernetes.io/

[41] *Prometheus*. Accessed: Dec. 27, 2024. [Online]. Available: https://prometheus.io/

[42] X. Foukas, N. Nikaein, M. M. Kassem, M. K. Marina, and K. Kontovasilis, "Demo: FlexRAN: A software-defined RAN platform," in *Proc. 23rd Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2017, pp. 465–467.

[43] P. Dinh, M. Ghoshal, Y. Han, Y. Feng, D. Koutsonikolas, and J. Widmer, "Demystifying resource allocation policies in operational 5G mmWave networks," *IEEE Trans. Netw.*, vol. 33, no. 2, pp. 745–760, Apr. 2025.

[44] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, vol. 1. Cambridge, MA, USA: MIT Press, 1998.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on September 25,2026 at 06:03:20 UTC from IEEE Xplore. Restrictions apply.